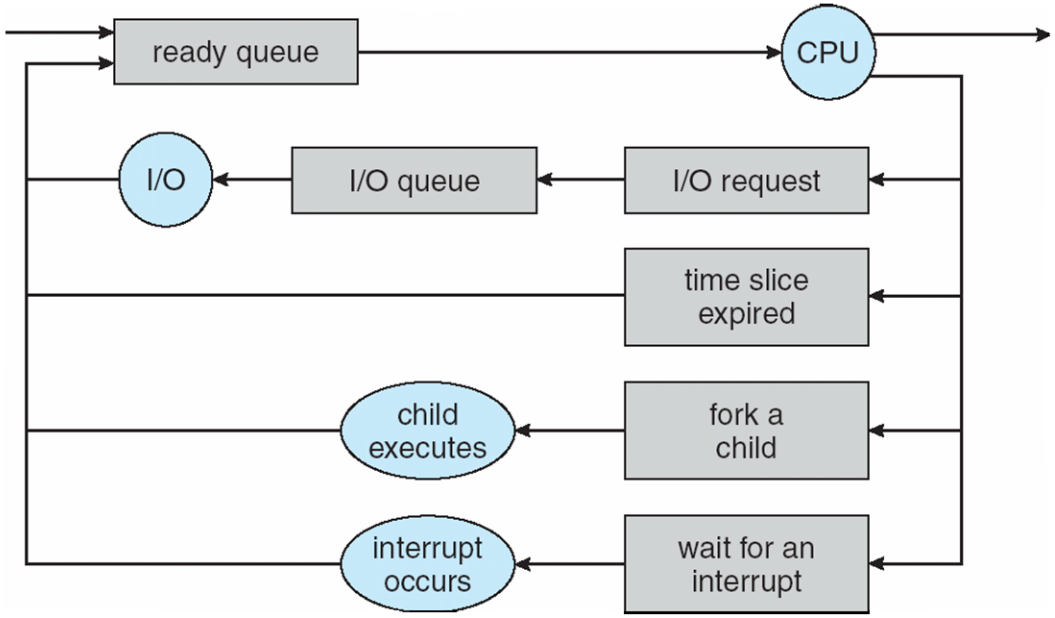
CPU Scheduling

- **Basic Concepts**
- **Scheduling Criteria**
- **Scheduling Algorithms**
  - **First-Come-First-Served**
  - **Shortest-Job-First, Shortest-remaining-Time-First**
  - **Priority Scheduling**
  - **Round Robin**

CPU Scheduling



- How is the OS to decide which of several tasks to take off a queue?
- Scheduling: deciding which process are assigned to cpu.

CPU Scheduling

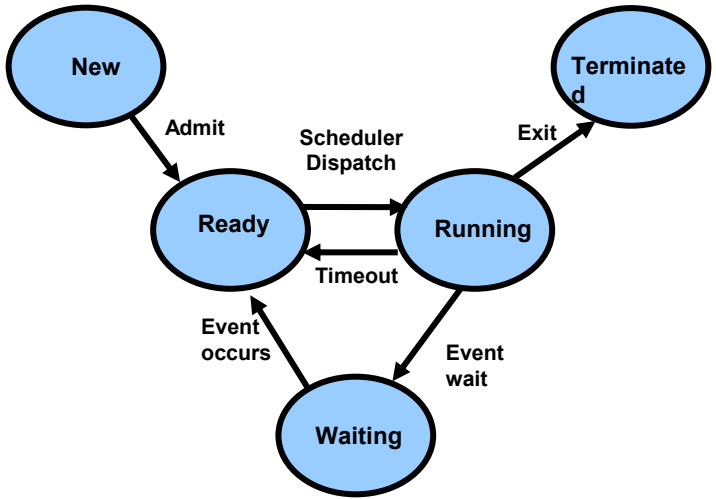
- **The *CPU scheduler* or *short-term scheduler* will select one of the processes in the ready queue for execution.**

- **Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:**
  - **switching context**
  - **switching to user mode**
  - **jumping to the proper location in the user program to restart that program**

When to Schedule?

Under a simple process state transition model, CPU scheduler could be invoked at four different points:

1. **When a process switches from the running state to the waiting state.**
2. **When a process switches from the running state to the ready state.**
3. **When a process switches from the waiting state to the ready state.**
4. **When a process terminates.**



Non-preemptive vs. Preemptive Scheduling

- **Under *non-preemptive scheduling*, each running process keeps the CPU until it completes or it switches to the waiting state (1 and 4 are non Preemptive Scheduling).**

- **Under *preemptive scheduling,* a running process may be also forced to release the CPU even though it is neither completed nor blocked. (2 and 3 are Preemptive Scheduling).**

Scheduling Criteria

- **Several criteria can be used to compare the performance of scheduling algorithms**
  - **CPU utilization – keep the CPU as busy as possible**
  - **Throughput – # of processes that complete their execution per time unit**
  - **Turnaround time – amount of time to execute a particular process**
  - **Waiting time – amount of time a process has been waiting in the ready queue**
  - **Response time – amount of time it takes from when a request was submitted until the first response is produced, not the complete output.**

Optimization Criteria

- Maximize the CPU utilization
- Maximize the throughput
- Minimize the (average) turnaround time
- Minimize the (average) waiting time
- Minimize the (average) response time

- **Arrival Time: Time at which the process arrives in the ready queue.**
- **Completion Time: Time at which process completes its execution.**
- **Burst Time: Time required by a process for CPU execution.**
- **Turn Around Time: Time Difference between completion time and arrival time.**

Turn Around Time = Completion Time – Arrival Time

- **Waiting Time(W.T): Time Difference between turnaround time and burst time. Waiting Time = Turn Around Time – Burst Time**

First-Come, First-Served (FCFS) Scheduling

- Single FIFO ready queue
- Non preemptive
  - Not suitable for timesharing systems
- Simple to implement and understand
- Average waiting time dependent on the order processes enter the system

First-Come, First-Served (FCFS) Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
  The *Gantt Chart* for the schedule:



- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0+24+27)/3 = 17

FCFS Scheduling (Cont.)

- Suppose that the processes arrive in the order P_2 , P_3 , P_1

- The Gantt chart for the schedule:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- *Problems:*
  - *Convoy effect* (short processes behind long processes)
  - Non-preemptive -- not suitable for time-sharing systems

Problem 1

Q. Consider the following processes with burst time (CPU Execution time). Calculate the average waiting time and average turnaround time?

Process id	Arrival time	Burst time/CPU execution time
P1	0	2
P2	1	3
P3	2	5
P4	3	4
P5	4	6

Problem 2

Consider the processes P1, P2, P3, P4 given in the below table, arrives for execution in the same order, with given Arrival Time and Burst Time.

PROCESS	ARRIVAL TIME	BURST TIME
P1	0	8
P2	1	4
P3	2	9
P4	3	5

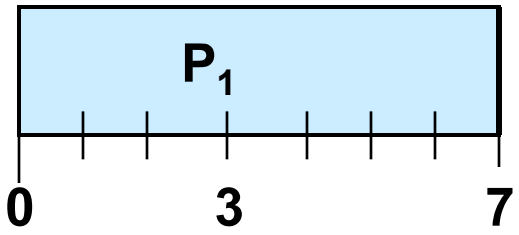
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. The CPU is assigned to the process with the smallest CPU burst (FCFS can be used to break ties).
- Two schemes:
  - nonpreemptive
  - preemptive – Also known as the Shortest-Remaining-Time-First (SRTF).
- Non-preemptive SJF is *optimal* if all the processes are ready simultaneously– gives minimum average waiting time for a given set of processes.
- SRTF is *optimal* if the processes may arrive at different times

Example for Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

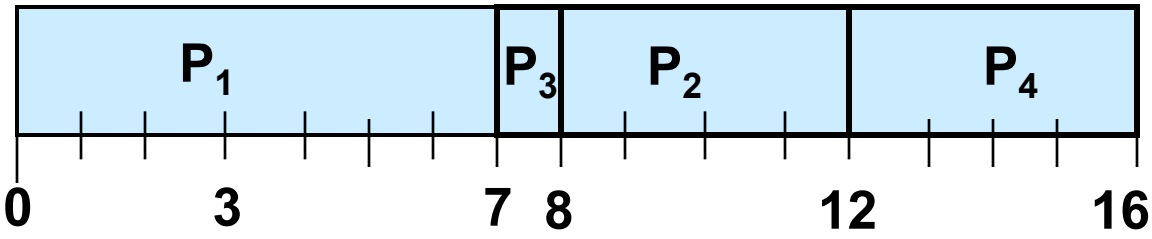


- At time 0, P_1 is the only process, so it gets the CPU and runs to completion

Example for Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- Once P_1 has completed the queue now holds P_2, P_3 and P_4



- P_3 gets the CPU first since it is the shortest. P_2 then P_4 get the CPU in turn (based on arrival time)
- Avg waittime = (0+8+7+12)/4 = 6.75

Estimating the Length of Next CPU Burst

- **Problem with SJF: It is very difficult to know exactly the length of the next CPU burst.**

Problem-1

Consider the set of 5 processes whose arrival time and burst time are given below-

Process Id	Arrival time	Burst time
P1	3	**1**
P2	1	**4**
P3	4	**2**
P4	0	**6**
P5	**2**	**3**

Problem-2

Consider the processes P1, P2, P3 given in the below table, arrives for execution in the same order, with Arrival Time 0, and given Burst Time,

PROCESS	ARRIVAL TIME	BURST TIME
P1	0	**24**
P2	0	**3**
P3	**0**	**3**

Problem-3

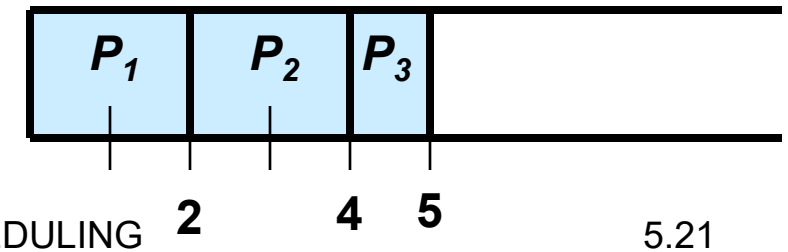
Consider the set of 3 processes whose arrival time and burst time are given below-
If the CPU scheduling policy is SRTF, calculate the average waiting time and average turn around time.

Process Id	Arrival time	Burst time
P1	0	9
P2	1	4
P3	2	9

Example for Preemptive SJF (SRTF)

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

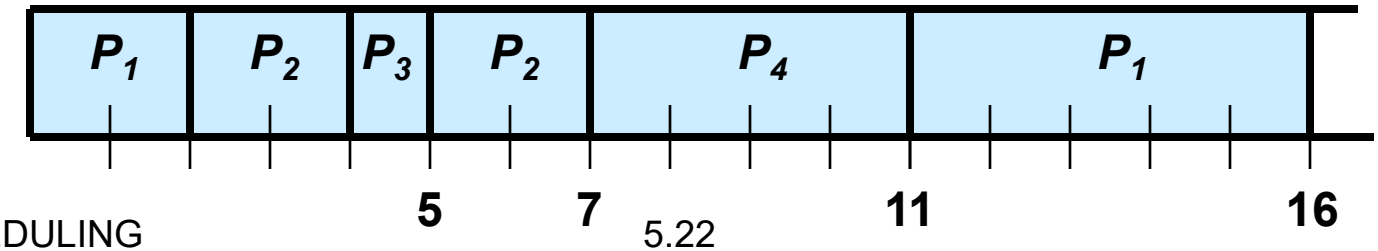
- Time 0 – P_1 gets the CPU Ready = [(P_1,7)]
- Time 2 – P_2 arrives – CPU has P_1 with time=5, Ready = [(P_2,4)] – P_2 gets the CPU
- Time 4 – P_3 arrives – CPU has P_2 with time = 2, Ready = [(P_1,5),(P_3,1)] – P_3 gets the CPU



Example for Preemptive SJF (SRTF)

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

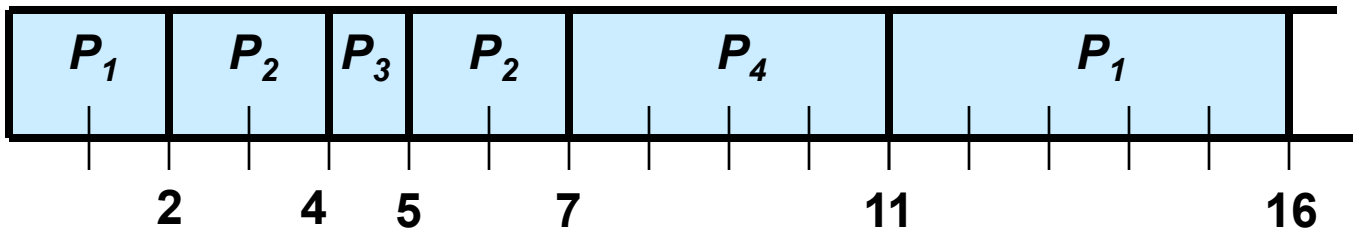
- Time 5 – P_3 completes and P_4 arrives - Ready = [(P_1,5),(P_2,2),(P_4,4)] – P_2 gets the CPU
- Time 7 – P_2 completes – Ready = [(P_1,5),(P_4,4)] – P_4 gets the CPU
- Time 11 – P_4 completes, P_1 gets the CPU



Example for Preemptive SJF (SRTF)

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Average waiting time = (9 + 1 + 0 +2)/4 = 3

EXAMPLE-1

Consider the set of 5 processes whose arrival time and burst time are given below-
If the CPU scheduling policy is SJF pre-emptive, calculate the average waiting time and average turnaround time.

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

EXAMPLE-2

Consider the set of 6 processes whose arrival time and burst time are given below-
If the CPU scheduling policy is shortest remaining time first, calculate the average waiting time and average turnaround time.

Process Id	Arrival time	Burst time
P1	0	**7**
P2	1	**5**
P3	2	**3**
P4	3	**1**
P5	4	**2**
P6	**5**	**1**

EXAMPLE-3

Consider the set of 3 processes whose arrival time and burst time are given below- If the CPU scheduling policy is SRTF, calculate the average waiting time and average turn around time.

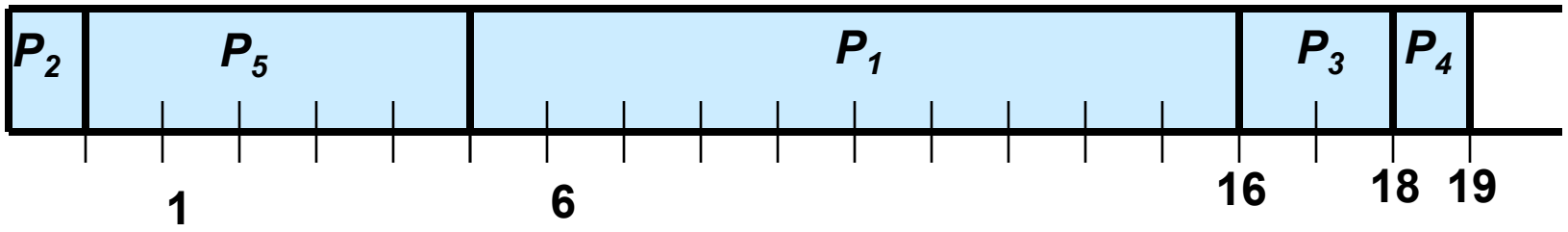
Process Id	Arrival time	Burst time
P1	0	9
P2	1	4
P3	2	9

Priority-Based Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority
  (smallest integer ≡ highest priority).
  - Preemptive
  - Non-preemptive

Example for Priority-based Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



Problem-1

Consider the set of 5 processes whose arrival time and burst time are given below-
If the CPU scheduling policy is priority non-preemptive, calculate the average waiting time and average turnaround time. (*Higher number represents higher priority*)

Process Id	Arrival time	Burst time	Priority
P1	0	4	**2**
P2	1	3	**3**
P3	2	1	**4**
P4	3	5	**5**
P5	**4**	**2**	**5**

Priority-Based Scheduling (Cont.)

- **Problem: Indefinite Blocking (or Starvation) – low priority processes may never execute.**
- **One solution: *Aging* – as time progresses, increase the priority of the processes that wait in the system for a long time.**
- **Priority Assignment**
  - **Internal factors: timing constraints, memory requirements, the ratio of average I/O burst to average CPU burst….**
  - **External factors: Importance of the process, financial considerations, hierarchy among users…**

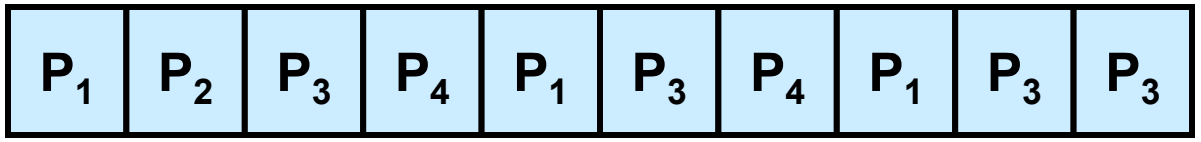
Round Robin (RR) Scheduling

- Each process gets a small unit of CPU time (*time quantum*). After this time has elapsed, the process is preempted and added to the end of the ready queue.
- Newly-arriving processes (and processes that complete their I/O bursts) are added to the end of the ready queue
- If there are *n* processes in the ready queue and the time quantum is *q*, then no process waits more than (*n*-1)*q* time units.
- Performance
  - *q* large $\Rightarrow$ *FCFS*
  - *q* small $\Rightarrow$ *Processor Sharing* (The system appears to the users as though each of the *n* processes has its own processor running at the $(1/n)^{th}$ of the speed of the real processor)

Example for Round-Robin

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

- The Gantt chart: (Time Quantum = 20)

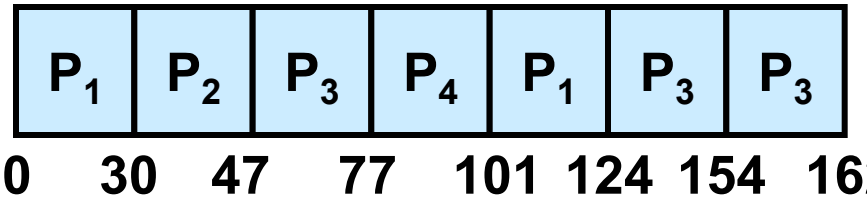


- Average wait time = (81+20+94+97)/4 = 73
- Typically, higher average turnaround time (amount of time to execute a particular process) than SJF, but better *response time* (amount of time it takes from when a request was submitted until the first response is produced).

Example for Round-Robin

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

- The Gantt chart: (Time Quantum = 30)



- Average wait time = (71+30+94+77)/4 = 68

- When Time Quantum = 10 get average wait time = (91+40+94+77)/4 = 75.5

Problem-1

Consider the set of 5 processes whose arrival time and burst time are given below-

If the CPU scheduling policy is Round Robin with time quantum = 2 unit, calculate the average waiting time and average turnaround time.

Process Id	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

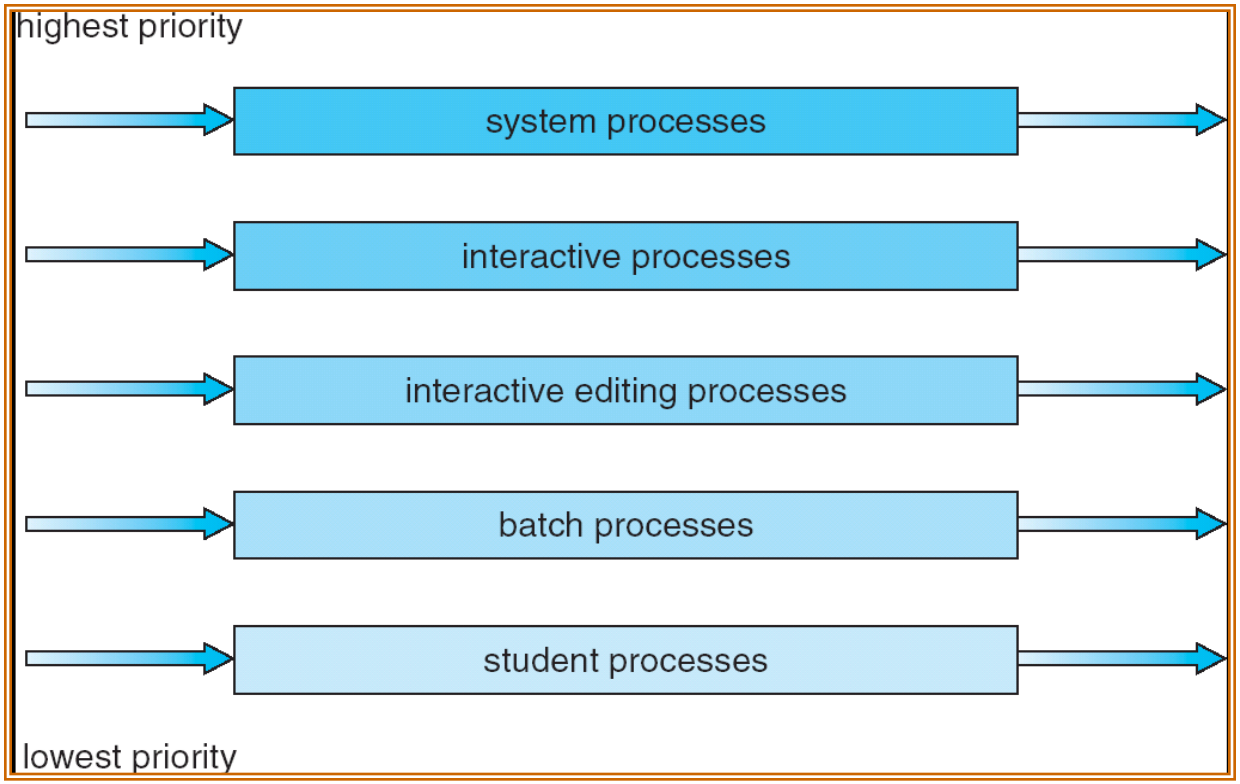
Problem-2

Consider the set of 6 processes whose arrival time and burst time are given below-
If the CPU scheduling policy is Round Robin with time quantum = 2, calculate the *average waiting time* and *average turnaround time.*

Process Id	Arrival time	Burst time
P1	0	4
P2	1	5
P3	2	2
P4	3	1
P5	4	6
P6	6	3

Multilevel Queue

- **Sometimes different processes can be partitioned into groups with different properties.**
- **Ready queue is partitioned into separate queues: Example, a queue for foreground (interactive) and another for background (batch) processes; or:**



Multilevel Queue Scheduling

- Each queue may have has its own scheduling algorithm: Round Robin, FCFS, SJF…

- In addition, (meta-)scheduling must be done between the queues.
  - Fixed priority scheduling (i.e. serve first the queue with highest priority). Problems?
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; for example, 50% of CPU time is used by the highest priority queue, 20% of CPU time to the second queue, and so on..
  - Also, need to specify which queue a process will be put to when it arrives to the system and/or when it starts a new CPU burst.

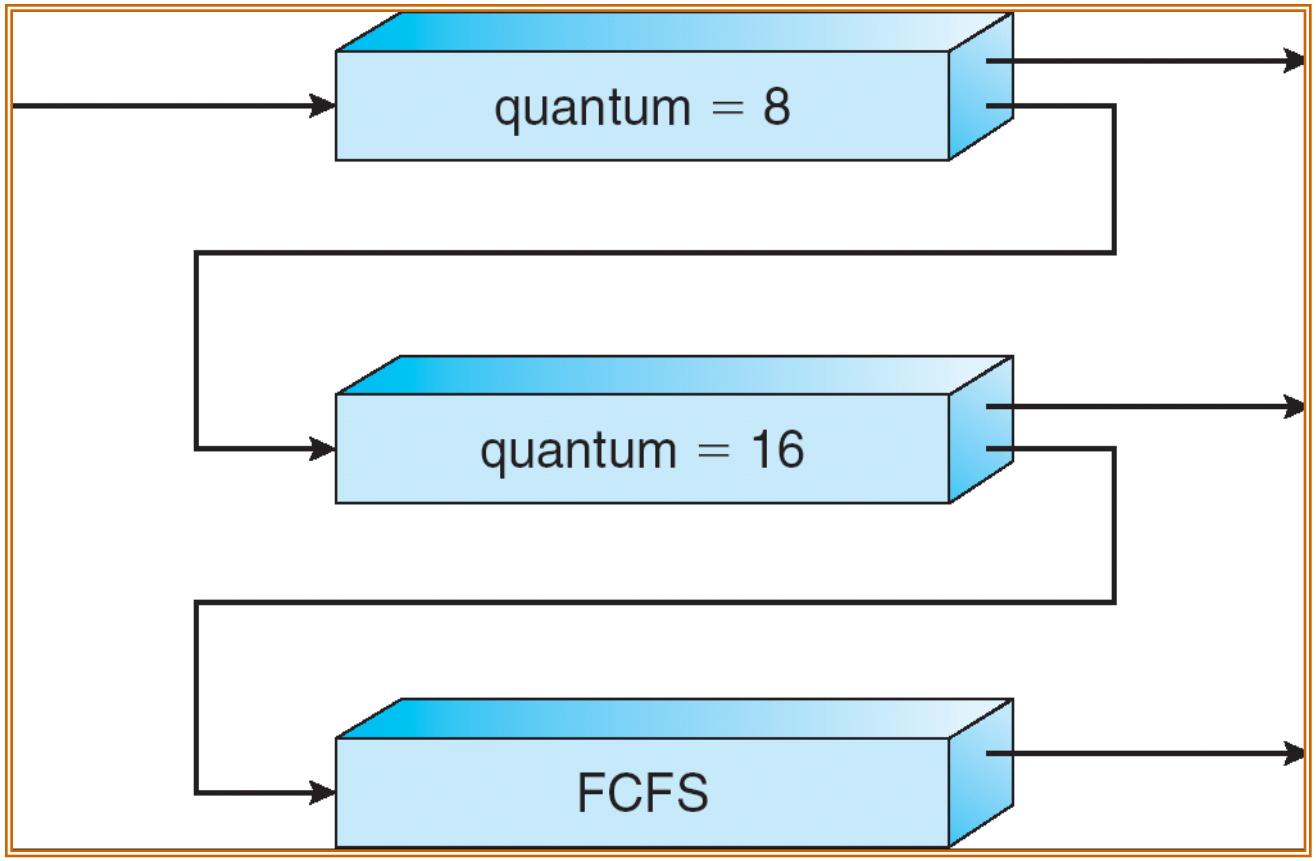
Multilevel Feedback Queue

- In a multi-level queue-scheduling algorithm, processes are permanently assigned to a queue.
- Idea: Allow processes to move among various queues.
- Examples
  - If a process in a queue dedicated to interactive processes consumes too much CPU time, it will be moved to a (lower-priority) queue.
  - A process that waits too long in a lower-priority queue may be moved to a higher-priority queue.

Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR - time quantum 8 milliseconds
  - Q_1 – RR - time quantum 16 milliseconds
  - Q_2 – FCFS
- Q_i has higher priority than Q_{i+1}
- Scheduling
  - A new job enters the queue Q_0. When it gains CPU, the job receives 8 milliseconds. If it does not finish in 8 milliseconds, the job is moved to the queue Q_1.
  - In queue Q_1 the job receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to the queue Q_2.

Multilevel Feedback Queue



Multilevel Feedback Queue

- **Multilevel feedback queue scheduler is defined by the following parameters:**
  - **number of queues**
  - **scheduling algorithms for each queue**
  - **method used to determine when to upgrade a process**
  - **method used to determine when to demote a process**
  - **method used to determine which queue a process will enter when that process needs service**

- **The scheduler can be configured to match the requirements of a specific system.**